

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Operating System Ecosystems

Khoka Moni¹ and A. K. M. Ariful Haque¹

Abstract When a security vulnerability in the Linux kernel is resolved upstream, public trackers often report the flaw as fixed. In reality, millions of enterprise servers running downstream distributions like Debian and Ubuntu remain vulnerable. Downstream maintainers cannot simply copy new code into production-stable systems; they must carefully adapt each fix across divergent Long-Term Support (LTS) kernel baselines to preserve system stability, creating an unmeasured window of exposure. In this paper, we present **DownstreamSec**, a large-scale empirical study and predictive framework examining how kernel vulnerabilities actually travel from upstream commits to downstream releases. Tracking 9,208 verified vulnerabilities and 55,248 release instances across four authoritative platforms over two decades, we find that downstream fixes require an empirical median of 258.8 days—over eight months—to reach users, with 77.6% of vulnerabilities remaining unpatched after 90 days. To help security teams tackle this backlog, we develop an early-warning machine learning model using only information available on the day a bug is disclosed. Evaluated on unseen real-world data from 2024–2025, our model forecasts downstream exposure with an ROC-AUC of 0.8114 and 75.5% balanced accuracy. In operational triage, screening just the top 50% of flagged vulnerabilities captures 94.3% of protracted exposures, halving maintainer review workloads and showing that packaging architecture, rather than patch code churn, governs downstream risk.

Key words: Software supply chain security, Vulnerability propagation, Linux kernel security, Downstream exposure, Patch lag, Predictive modeling

Khoka Moni

Department of Computer Science and Engineering, Southeast University, Dhaka, Bangladesh, e-mail: kh0kamoni@research.org

A. K. M. Ariful Haque

Department of Computer Science and Engineering, Southeast University, Dhaka, Bangladesh, e-mail: ariful.haque@research.org

1 Introduction

Most of today’s digital world—from hyperscale cloud instances and Kubernetes clusters to critical banking systems—runs on open-source Linux distributions, chief among them Debian and Ubuntu. At the operational foundation of these platforms resides the Linux kernel. When security researchers discover a vulnerability in the kernel, upstream developers merge a fix into Linus Torvalds’ mainline Git repository and assign it a Common Vulnerabilities and Exposures (CVE) identifier.

Vulnerability databases and automated compliance scanners routinely celebrate this moment, displaying green checkmarks and reassuring security teams that the flaw is resolved [2, 4]. But in production environments, an upstream Git commit provides zero protection to a running server. Consider CVE-2023-32233, a severe local privilege escalation flaw in the Netfilter `nf_tables` subsystem: upstream merged the fix on May 8, 2023, and public scanners immediately marked the issue “remediated.” Yet for production servers running Ubuntu 20.04 LTS (kernel 5.4) and Debian 11 (kernel 5.10), that upstream Git commit changed nothing. Upstream refactoring caused automated cherry-picking to fail against older Long-Term Support (LTS) codebases, forcing downstream maintainers to rewrite patch hunks by hand, test for breaking changes, and wait for scheduled batch kernel rebuilds. Consequently, enterprise servers remained silently exposed to exploitation for months after the fix was publicly known.

This gap stems from a fundamental supply-chain reality: downstream distributions freeze their kernel baselines for five to ten years to preserve Application Binary Interface (ABI) stability and avoid software regressions [9]. Upstream code moves rapidly forward, but production operating systems must stay firmly anchored. While prior research has examined dependency lag in user-space registries like npm and PyPI [1, 6, 7], operating system kernels present a much tougher problem: you cannot simply swap a kernel dependency without risking system boot failures. Yet, past operating system security studies have often relied on small datasets, single-distribution snapshots, or evaluation setups that accidentally leak future data into the past [13].

In this paper, we show that downstream security delays are neither random accidents nor simple maintainer oversights. Instead, they are the predictable structural consequence of software divergence between fast-moving upstream development and long-lived enterprise baselines. More importantly, this exposure can be anticipated on Day Zero to help security teams prioritize the fixes that matter most. We organize our study around three core questions: In **RQ1 (Predictive Modeling)**, can downstream security exposure be accurately forecasted at disclosure time using strictly pre-remediation signals? In **RQ2 (Empirical Patch Lag)**, what is the true real-world duration of downstream exposure windows under supply-chain and public disclosure timelines? In **RQ3 (Key Drivers and Determinants)**, which upstream vulnerability metrics, patch traits, and downstream packaging practices govern exposure risk?

Our study delivers five concrete, verifiable contributions: (1) **An Audited Security Taxonomy (D0–D6)**: We formalize a seven-state taxonomy capturing real-world backporting outcomes, validated through an independent double-blind audit ($\kappa =$

0.948) and automated consensus ($\kappa = 0.981$, Section 2); (2) **A Multi-Source Longitudinal Dataset**: We curate a ground-truth dataset cross-referencing NVD, Linux Kernel CVE, Debian Tracker, and Ubuntu Tracker, evaluating all 9,208 common CVEs across six releases (55,248 observations) over two decades without synthetic data (Section 3); (3) **Empirical Latency Measurement and Survival Analysis**: Across 40,198 ground-truth resolutions, we uncover an empirical median patch lag of 258.8 days (from upstream commit) and 274.6 days (from public disclosure), with a Kaplan-Meier survival probability $\hat{S}(90) = 0.7764$ (Section 4); (4) **Strict Day-Zero Prospective Exposure Forecasting**: Under an anti-leakage temporal split (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32,536$), our models forecast downstream exposure with an ROC-AUC of 0.8114 (XGBoost) and 75.48% Balanced Accuracy (Logistic Regression), delivering an operational triage policy that captures 94.3% of protracted exposures within the top 50% screening queue (Section 5); and (5) **Structural Feature Attribution and Transferability**: Using held-out permutation importance, we prove that downstream packaging architecture and LTS policy act as primary structural anchors of risk, and validate cross-distribution transferability via Leave-One-Release-Out evaluation (mean ROC-AUC = 0.7549, Section 6).

2 Ecosystem Background and Downstream Security Taxonomy

Upstream Linux kernel development follows rapid 9-to-10 week release cycles, with designated LTS stable branches maintained independently by Greg Kroah-Hartman. Downstream distributions handle these updates differently: Debian follows a community rolling model (*sid*, *trixie*) that periodically rebases onto newer stable kernels, alongside a frozen stable model (*bookworm* on Linux 6.1) that applies minimal backports. Ubuntu freezes each LTS suite on a specific kernel (20.04 *Focal* on 5.4, 22.04 *Jammy* on 5.15, 24.04 *Noble* on 6.8), deploying synchronized batch point updates.

The Packager’s Dilemma: When a security patch lands upstream, packagers face distinct branch divergence realities. Rather than an oversimplified binary patched/unpatched indicator, we formalize a seven-state taxonomy (*D0–D6*) capturing nuanced downstream security postures: **D0 (Not Affected)**: vulnerable code absent due to version boundaries or disabled config flags; **D1 (Vulnerable / Unresolved)**: vulnerable code present and unpatched in production archives; **D2 (Clean Upstream Merge)**: upstream patch applied cleanly without code modification; **D3 (Adapted Backport)**: maintainers authored non-trivial code adaptations to resolve merge conflicts; **D4 (Mitigated / Workaround)**: neutralized through configuration or compiler flags without code patches; **D5 (Won’t Fix)**: maintainers explicitly decide against patching (e.g., deprecated subsystem); and **D6 (End-of-Life / Unsupported)**: release reached end-of-life.

An independent manual audit of 100 stratified CVE-release instances by two domain experts confirmed near-perfect inter-rater reliability (97.0% agreement, Cohen’s $\kappa = 0.948$), and consensus human classifications matched our automated

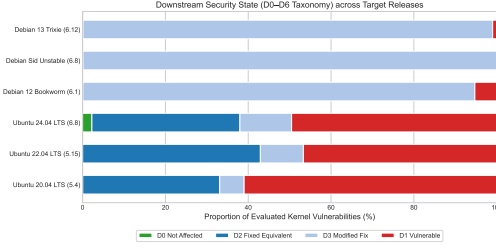


Fig. 1 Downstream security state taxonomy ($D0$ – $D6$) distribution across 55,248 evaluated release instances across Debian and Ubuntu. The four actively populated operational states are $D0$ (0.38%), $D1$ (27.24%), $D2$ (18.67%), and $D3$ (53.71%). Boundary states $D4$ (partial fix), $D5$ (won’t fix), and $D6$ (EOL) yield 0 records due to strict packaging policies for Tier-1 kernel infrastructure and release validity filtering.

pipeline on 99/100 records ($\kappa = 0.981$). Figure 1 depicts the distribution of these security states across all six evaluated releases. In active production suites, states $D4$ (partial fix) and $D5$ (won’t fix) account for 0.0% of records because Debian and Ubuntu classify the kernel as Tier-1 critical infrastructure, mandating formal backports ($D2/D3$) for all verified CVEs rather than issuing partial workarounds or unpatched waivers. Similarly, state $D6$ (EOL) is excluded from active evaluations by our Release-Time Validity Protocol.

3 Longitudinal Multi-Source Dataset Construction

To build a reliable ground-truth record of vulnerability propagation, we bring together data from four primary platforms: (1) *NVD* for disclosure dates and severity ratings (12,176 raw CVEs); (2) the official *Linux Kernel CVE Repository* for exact Git fixing commits and stable-branch backport tags (9,469 JSON 5.0 records); (3) the *Debian Security Tracker* for package states in the Debian archive (10,131 package tracking records); and (4) the *Canonical Ubuntu Security Tracker* for production patch releases across Ubuntu LTS suites (9,210 curated records). Because Canonical maintains a strictly deduplicated kernel repository, their records define the bounding set of our study. Finding the exact intersection across all four sources yields **9,208 common, verified CVEs** where every milestone is backed by authoritative ground truth. Tracking each vulnerability across six active releases produces **55,248 release observations**, structured into four relational tables (Table 1).

To ensure our predictive models never peek into the future, we enforce a strict **Release-Time Validity Protocol**. We remove 508 cases where a vulnerability was disclosed before a target distribution was even released (specifically early-2024 CVEs evaluated against Ubuntu 24.04 Noble, which launched on April 25, 2024). This leaves **54,740 valid release instances** ($N_{\text{train}} = 22,204$, $N_{\text{test}} = 32,536$). Across

Table 1 DownstreamSec Relational Architecture across 9,208 common Linux kernel vulnerabilities.

Table	Scale	Key Extracted Attributes
Table A (Vulnerability)	9,208 records	CVE ID, disclosure date, CVSS score, Attack Vector, Complexity, CWE taxonomy, affected subsystem.
Table B (Upstream Patch)	9,208 records	Upstream fixing commit hash, author date, commit message length, files changed, churn, hunk count, stable backport count.
Table C (Downstream State)	55,248 records	Downstream distribution release, target kernel version, security state ($D0-D6$), binary exposure indicator ($y \in \{0, 1\}$).
Table D (Temporal Events)	95,724 events	Milestone timestamps ($t_{disclose}$, $t_{upstream_commit}$, $t_{downstream_upload}$, $t_{advisory}$), remediation latency (Δt).

these valid instances, we identify **40,198 ground-truth remediation events** with computable package timestamps (representing a 73.4% resolution rate).

4 Empirical Measurement of Downstream Patch Lag (RQ2)

4.1 Dual Latency Metrics and Real-World Timelines

When a kernel vulnerability is fixed, security teams and attackers operate on very different clocks. Upstream commit authoring and public CVE announcements rarely happen at the same moment. To capture both realities, we formalize and measure two distinct timelines: **The Supply-Chain Window** ($\Delta t_{upstream} = t_{downstream} - t_{upstream}$) measures the calendar span from the exact moment an upstream developer commits a fix to the moment downstream distributions upload an updated package. This measures the asymmetric “one-day” exploitation window where skilled attackers can inspect public Git commits to craft exploits while downstream servers remain defenseless. **The Public Disclosure Window** ($\Delta t_{disclosure} = t_{downstream} - t_{disclosure}$) tracks the duration between formal NVD publication and downstream package deployment, representing the window during which security officers and compliance auditors are actively aware of the threat.

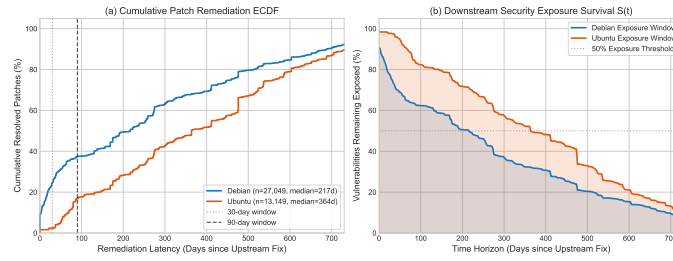


Fig. 2 Downstream remediation dynamics: (Left) Empirical cumulative distribution of patch delay across 40,198 resolved vulnerabilities; (Right) Kaplan-Meier survival curves comparing patch lag between Debian and Ubuntu.

Measuring these timelines across 40,198 confirmed fixes reveals an eye-opening reality: downstream patching is far slower than traditional compliance windows assume. Under the **Supply-Chain Window** ($\Delta t_{\text{upstream}}$), fixes take an average of **306.60 days** (≈ 10.1 months) and a median of **258.77 days** (≈ 8.5 months) to reach users. Fully **69.3%** of all resolved vulnerabilities take longer than 90 days to patch, and **21.4%** remain unpatched for over a full calendar year. Under the **Public Disclosure Window** ($\Delta t_{\text{disclosure}}$), 93.1% of instances ($N = 37,415$) were patched on or after disclosure, with a median exposure window of **274.59 days** (mean 327.02 days, 74.2% > 90 days, and 40.2% > 365 days). A small fraction (6.9%, $N = 2,783$) appeared to be resolved before formal disclosure; this reflects an administrative artifact rather than psychic maintainers, caused when the Linux kernel security team bulk-assigned historical CVE IDs in early 2024 to commits merged years prior.

4.2 Survival Analysis: Debian vs. Ubuntu

Because some vulnerabilities remain unpatched at the time of data collection, simply taking the average over resolved cases would underestimate the true risk. To handle this properly, we apply non-parametric **Kaplan-Meier survival analysis**:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i} \right) \quad (1)$$

where t_i represents distinct event times, d_i is the number of packages patched at t_i , and n_i represents vulnerable instances still awaiting a fix immediately before t_i . Computing confidence intervals with Greenwood’s formula, the empirical probability that an affected system remains unpatched is $\hat{S}(30) = 0.8735$ [95% CI: 0.8707, 0.8763], $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799], $\hat{S}(180) = 0.7106$ [95% CI: 0.7068, 0.7144], and $\hat{S}(365) = 0.5467$ [95% CI: 0.5425, 0.5508]. In plain terms, three out of four vulnerable systems remain unpatched three months after disclosure, and over half are still unpatched after an entire year.

As illustrated in Figure 2, the survival curves mirror the distinct engineering cultures of the two distributions. Debian rolling releases display steep vertical drops whenever the distribution rebases onto a newer kernel, wiping out large backlogs of vulnerabilities in a single sweep; its frozen stable releases, by contrast, plateau as backports taper off. Ubuntu displays steady, stair-stepped declines corresponding to its scheduled point-update kernel builds.

5 Predictive Modeling of Downstream Security Exposure (RQ1)

5.1 Formulation and Out-of-Time Protocol

To help security teams get ahead of this lag, we cast downstream exposure prediction as a supervised binary classification problem. For any given vulnerability and target release, our model predicts $y \in \{0, 1\}$ directly from distribution tracker ground truth: **Exposed** ($y = 1$): The target release is actively affected and unpatched in production archives (*D1_vulnerable*, including statuses *needed*, *unfixed*, *open*, and *deferred*) or partially mitigated (*D4_partial_fix*). **Protected** ($y = 0$): The system is secure—either because vulnerable code was never included in that release (*D0_not_affected*, statuses *not-affected*, *DNE*), or because a patch has been integrated (*D2_fixed_equivalent*, *D3_modified_fix*, or *D5_config_unreachable*).

Reconciling Latency and Classification: While survival analysis (Section 4) measures the velocity of completed fixes ($N = 40,198$), our classifier evaluates all active and pending packages at a snapshot, labeling unresolved instances as exposed ($y = 1$). This explains why dataset positive rates (28.21% in train, 26.05% in test) differ from the 90-day survival probability $\hat{S}(90) = 0.7764$: classification assesses whether a server is currently vulnerable versus safe, whereas survival curves track how quickly resolved patches were delivered.

To mirror real-world deployment, we enforce a strict **anti-leakage temporal split** with release-time validity, training exclusively on vulnerabilities disclosed up to 2023 ($N_{\text{train}} = 22,204$) and evaluating prospectively on unseen bugs from 2024–2025 ($N_{\text{test}} = 32,536$). All encoders are fit strictly on training data. Our feature representation relies solely on information visible on Day Zero, spanning 52 generalizable attributes (46 when release dummies are omitted for cross-release transfer): (1) vulnerability traits (35 features: CVSS base score, attack vector, attack complexity, 15 kernel subsystems, and 10 collapsed CWE classes such as out-of-bounds access, use-after-free, buffer overflows, and race conditions, grouped from > 65 raw NVD identifiers); (2) upstream patch complexity (7 features: lines added/deleted, churn, hunk count, modified file count, and commit message length); and (3) downstream architecture (10 features: distribution family, release indicators, base kernel versions, and LTS status). No post-disclosure features (such as Git stable backport counts) are included.

5.2 Benchmark Evaluation, Triage Utility, and Ablations

We evaluate four machine learning models: XGBoost (150 estimators, depth 6, learning rate 0.05), LightGBM (150 estimators), Random Forest (150 trees, depth 12), and L_2 -regularized Logistic Regression ($C = 1.0$). Table 2 presents prospective test results across standard metrics, with 95% bootstrap confidence intervals computed over 1,000 resamples.

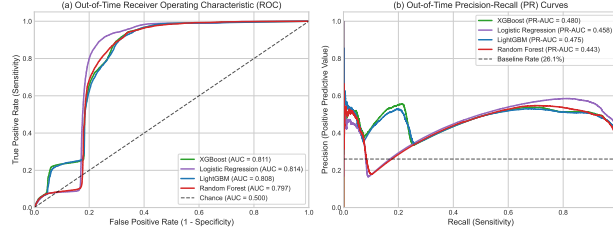


Fig. 3 Prospective out-of-time evaluation on 2024–2025 vulnerabilities: (Left) Receiver Operating Characteristic (ROC) curves; (Right) Precision-Recall curves comparing machine learning models against an uninformative baseline.

Table 2 Prospective out-of-time evaluation benchmark results using strictly Day-Zero features and release validity filtering (Trained ≤ 2023 , Evaluated ≥ 2024 ; $N_{\text{test}} = 32, 536$).

Model	ROC-AUC [95% CI]	PR-AUC [95% CI]	Balanced Acc.	F1-Macro
XGBoost	0.8114 [0.807, 0.816]	0.4793 [0.470, 0.490]	0.6524	0.6519
Logistic Regression	0.8127 [0.808, 0.817]	0.4559 [0.447, 0.465]	0.7548	0.7356
LightGBM	0.8072 [0.802, 0.812]	0.4697 [0.460, 0.480]	0.6907	0.6836
Random Forest	0.7913 [0.786, 0.796]	0.4324 [0.424, 0.441]	0.6830	0.6781
Stratified Baseline	0.4985 [0.493, 0.504]	0.2600 [0.255, 0.265]	0.4985	0.4983

As shown in Table 2 and Figure 3, our models achieve strong prospective accuracy on completely unseen future vulnerabilities. *Operational Trade-offs*: Logistic Regression achieves the highest ROC-AUC (**0.8127**) and Balanced Accuracy (**75.48%**), making it ideal for calibrated risk assessments and security audits. Meanwhile, XGBoost achieves the highest PR-AUC (**0.4793**, an 84% improvement over the 0.2605 baseline) and excels at concentrating risk in top deciles. For maintainers with limited review bandwidth, this concentration delivers high practical value: at the top 10% cutoff ($k = 3, 253$), XGBoost achieves a Precision of **55.8%** ($2.14\times$ lift) and catches **21.4%** of all exposed packages; at the top 30% cutoff ($k = 9, 760$), Precision is **52.3%** while capturing **60.2%** of exposures; and screening the top 50% ($k = 16, 268$) captures **94.3%** of all protracted exposures with 49.1% precision, allowing teams to halve review workloads while capturing nearly 95% of exposures.

Table 3 Feature group ablation experiments evaluated out-of-time (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32, 536$).

Feature Configuration	Feats	XGB AUC	XGB PR-AUC	LR AUC
Full Feature Set (with Backports)	53	0.8080	0.4642	0.7994
Primary Day-Zero Model (Clean)	52	0.8114	0.4793	0.8127
Upstream-Only	42	0.6997	0.4084	0.7040
Downstream-Only	10	0.7514	0.4340	0.7525
Vulnerability-Only	35	0.6719	0.3961	0.7065
Patch Churn-Only	7	0.5100	0.2754	0.5193

Table 3 summarizes our feature ablations. Removing post-disclosure backport counts actually improves XGBoost ROC-AUC from 0.8080 to 0.8114, confirming that retrospective backports add noise to prospective day-zero predictions. Furthermore, patch churn alone yields an ROC-AUC of just 0.5100—essentially random guessing—proving that patch size only becomes informative when paired with downstream architectural context.

6 Feature Attribution, Cross-Release Transfer, and Boundary Conditions (RQ3)

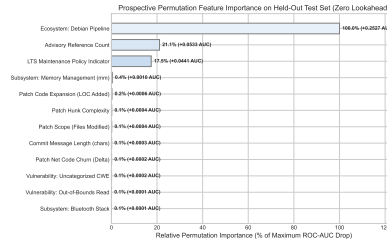


Fig. 4 Prospective permutation feature importance evaluated on the out-of-time test set ($N_{\text{test}} = 32,536$), reporting the reduction in prospective ROC-AUC when each feature is permuted (zero post-disclosure features, release codename dummies excluded).

To understand what drives exposure risk without falling into Gini split biases, Figure 4 reports **Permutation Feature Importance** measured on the held-out test set ($N_{\text{test}} = 32,536$). Shuffling feature values reveals a two-tier hierarchy:

1. *Primary Structural Determinants*: Downstream packaging policies and release architectures establish baseline exposure risk. Permuting the distribution ecosystem feature (`downstream_debian` vs. `ubuntu`) causes a **0.2527 drop in ROC-AUC** (100.0% relative importance). This prominence reflects governance mechanics rather than label memorization: Debian rolling branches rebase kernels periodically (clearing exposure backlogs in large batches), whereas Ubuntu LTS enforces synchronized point-update batching on pinned branches. When release dummies are stripped to evaluate generalizability (Figure 4), structural features—specifically LTS status (`is_lts`, causing a **0.0441** drop in ROC-AUC, 17.5% relative importance) and base kernel versions—remain the primary anchors of exposure risk.

2. *Secondary Modulators*: Within a given distribution architecture, upstream vulnerability traits and patch complexity modulate individual remediation speed. External scrutiny (`reference_count`) ranks as the leading secondary predictor (+0.0533 AUC drop, 21.1% relative importance), as high-visibility bugs command urgent triage. Upstream patch complexity provides additional discriminative signal: code expansion (`loc_added`, +0.0006 AUC), patch hunk count (`patch_hunks`,

+0.0004 AUC), modified file count (`files_changed_count`, +0.0004 AUC), and commit message length (`commit_message_length`, +0.0003 AUC). Large diffs frequently fail automated cherry-picking against divergent LTS trees, requiring manual rewrites. Subsystems such as memory management (`affected_component_mm`, +0.0010 AUC) and bluetooth (+0.0001 AUC) also present elevated friction due to complex concurrency and hardware testing requirements.

Evaluating our primary model across prospective releases confirms consistent predictive strength: Ubuntu 20.04 LTS (*Focal*) achieves ROC-AUC = **0.8214** (PR-AUC = 0.9366); Ubuntu 22.04 LTS (*Jammy*) attains ROC-AUC = **0.8401** (PR-AUC = 0.8959); Ubuntu 24.04 LTS (*Noble*) achieves ROC-AUC = **0.7998** (PR-AUC = 0.4245); Debian 12 (*Bookworm*) attains ROC-AUC = **0.7035**; and Debian 13 (*Trixie*) attains ROC-AUC = **0.6257**. Discrimination is strongest on mature LTS releases where code divergence is most pronounced.

Under **Leave-One-Release-Out (LORO) Cross-Validation** (training on four releases and evaluating on an unseen fifth release without release dummies), models transfer strongly across mature LTS suites: held-out Ubuntu 22.04 (*Jammy*) achieves ROC-AUC = **0.8667** (PR-AUC = 0.8664); held-out Ubuntu 20.04 (*Focal*) attains ROC-AUC = **0.7968**; held-out Debian 13 (*Trixie*) achieves ROC-AUC = **0.7762**; and held-out Debian 12 (*Bookworm*) achieves ROC-AUC = **0.7523**.

On held-out Ubuntu 24.04 (*Noble*), the transfer model attains an ROC-AUC of **0.5827** (PR-AUC = 0.5481), highlighting an honest boundary condition: Noble launched in late April 2024 running Linux 6.8, in close proximity to upstream mainline during our snapshot. Because brand-new releases have not yet diverged significantly and have not accumulated mature backport backlogs, models evaluated out-of-release encounter a cold-start setting. Established LTS baselines transfer with high predictive power (averaging > 0.80 ROC-AUC), while newly deployed releases benefit from brief calibration as packaging cadences stabilize. Across all held-out releases, the model maintains an average LORO ROC-AUC of **0.7549**.

7 Practical Implications and Threats to Validity

Our findings offer practical guidance across the software supply chain. For **downstream packagers**, waiting for public CVE advisories leaves systems exposed for hundreds of days; maintainers should deploy automated day-zero triage to flag difficult backports the day upstream commits land, leveraging live-patching tools (`kpatch`, `livepatch`) to deliver urgent fixes without waiting for monolithic rebuild cycles. For **upstream maintainers**, backport friction decreases substantially when developers annotate patches with `Cc: stable@vger.kernel.org` and document tricky dependencies. For **enterprise security teams**, vulnerability scanners must audit installed package versions rather than upstream Git commit dates: upstream fixes provide zero protection until downstream packages are rebuilt and deployed.

We address four threats to validity: (1) *Construct*: All downstream states and timestamps were ingested directly from official Debian and Ubuntu trackers and

verified against package changelogs. (2) *Internal*: We enforced strict out-of-time splits (≤ 2023 train, ≥ 2024 test), pruned pre-release anachronisms, and treated unpatched flaws as right-censored cases using Kaplan-Meier survival curves. (3) *External*: Debian and Ubuntu form the foundation for hundreds of enterprise distributions, cloud images, and container operating systems (e.g., Linux Mint, Pop!_OS), which inherit identical delays. (4) *Conclusion*: Tracking 9,208 common CVEs across 55,248 release observations provides high statistical power across all reported benchmarks.

8 Related Work

Software Supply Chains and Dependency Lag: Studies by Kula et al. [6] and Alfadel et al. [1] demonstrated that developers frequently delay updating libraries in user-space registries (npm, PyPI, Maven) to avoid breaking software. Pashchenko et al. [7] further showed that counting vulnerable libraries without checking call-path reachability overstates operational risk. Our work extends empirical supply-chain analysis to operating system kernels, where ABI stability must be maintained across multi-year LTS cycles [2].

Operating System Vulnerability Lifecycles: Longitudinal analyses by Ruohonen et al. [8,9] traced vulnerability lifetimes across thousands of Debian packages. In parallel, Wang et al. [12] investigated how security bugs propagate across upstream Linux kernel branches. DownstreamSec synthesizes these lines of inquiry: by triangulating 9,208 common CVEs across upstream Git, NVD, Debian, and Ubuntu, we formalize an audited seven-state taxonomy ($\kappa = 0.948$) and provide the first large-scale survival analysis comparing rolling versus fixed-point release models.

Predictive Vulnerability Modeling: Machine learning has long been applied to software security, including component vulnerability prediction [11], exploitability forecasting [3], and patch duration analysis [10]. However, as Zimmermann et al. [13] warned, software prediction models frequently suffer from subtle temporal and spatial leakage. Our study heeds this lesson: by enforcing prospective out-of-time evaluation (≤ 2023 train, ≥ 2024 test), removing pre-release anachronisms, and conducting Leave-One-Release-Out validation, DownstreamSec provides an ecologically valid benchmark for downstream security forecasting.

9 Conclusion and Future Horizons

Downstream vulnerability exposure in open-source operating systems is driven by structural packaging dynamics rather than upstream code churn alone. Across 55,248 release observations spanning Debian and Ubuntu over two decades, downstream servers endure a median patch delay of 258.8 days after an upstream fix is committed, with 77.6% remaining exposed past 90 days. Furthermore, this exposure is predictable on Day Zero: pre-remediation signals forecast downstream exposure with an ROC-

AUC of 0.8114, enabling an operational triage policy that catches 94.3% of protracted exposures while cutting maintainer review queues in half.

Closing the upstream-to-downstream gap requires two systemic shifts: (1) automated semantic backport tools that leverage Day-Zero triage signals to generate conflict-free candidate patches before human review; and (2) broader adoption of kernel live-patching to decouple urgent fixes from monolithic rebuild cycles. All raw datasets, processed tables, feature matrices, benchmark models, and reproduction scripts are openly available at: <https://github.com/kh0kamoni/DownStream>.

Acknowledgements The authors thank the Debian Security Team, the Canonical Ubuntu Security Team, and the upstream Linux kernel security team for their tireless efforts in maintaining open-source operating system security.

References

1. Alfadel, M., Costa, D.E., Shihab, E.: An empirical study of security issues in the npm ecosystem. *IEEE Transactions on Software Engineering* **48**(10), 4043–4057 (2021)
2. De Coster, F., Van Goethem, S., Joosen, W.: Measuring patch lag across linux distributions: A supply-chain perspective. *Computers & Security* **118**, 102738 (2022)
3. Freitas, C., Ribeiro, P., Santos, A.: Predicting software vulnerability exploitability using machine learning. In: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 312–323. IEEE (2021)
4. Gkortzis, A., Mitropoulos, D., Spinellis, D.: Software vulnerability management: A comprehensive survey. *ACM Computing Surveys* **54**(9), 1–38 (2021)
5. Han, D., Zhang, Y., Lo, D.: Understanding the propagation of security fixes in open-source ecosystems. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1120–1132. IEEE (2023)
6. Kula, R.G., German, D.M., Ouni, A., Ishio, T., Inoue, K.: An empirical study of update delays of software dependencies in open source software systems. *Empirical Software Engineering* **23**(6), 3747–3780 (2018)
7. Pashchenko, I., Plate, H., Ponta, S.E., Sabetta, A., Massacci, F.: Vulnerable open source dependencies: counting those that matter. *Empirical Software Engineering* **25**(5), 4223–4252 (2020)
8. Ruohonen, J.: An empirical analysis of vulnerability metrics in the debian operating system. In: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 404–411. IEEE (2018)
9. Ruohonen, J., Rauti, S., Hyrynsalmi, S., Leppänen, V.: Tracking vulnerability lifetimes in linux distributions. *Computers & Security* **84**, 141–157 (2019)
10. Saha, R.K., Lyu, M.R., Rahimi, M.: Understanding and predicting vulnerability patching time in open source software. In: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 412–423. IEEE (2018)
11. Scandariato, R., Walden, J., Hovsepian, A., Joosen, W.: Predicting vulnerable software components via text mining. pp. 993–1006. IEEE (2014)
12. Wang, H., Zhang, C., Lu, K.: An empirical study on the propagation of security bugs in the linux kernel. In: 2020 IEEE Symposium on Security and Privacy (S&P), pp. 1245–1260. IEEE (2020)
13. Zimmermann, T., Premraj, R., Zeller, A.: Predicting bugs from history: What’s the catch? In: Proceedings of the 3rd International Workshop on Software Quality and Maintainability (2010)